

Week 3&4

Creating Freestyle Job and Pipeline Job in Jenkins

1. Aim

To understand how to create and configure Freestyle and Pipeline jobs in Jenkins for automating software build and deployment tasks.

2. Introduction to Jenkins

Jenkins is an open-source automation server used to implement Continuous Integration and Continuous Deployment (CI/CD). It allows users to create different types of jobs to automate tasks like building, testing, and deploying applications.

3. What is a Freestyle Job?

A Freestyle job is the simplest Jenkins job type. It allows you to configure build steps through the Jenkins UI without writing code. It is mainly used for basic automation tasks.

4. Steps to Create a Freestyle Job

1. Open Jenkins dashboard in browser (<http://<server-ip>:8080>)
2. Click on 'New Item'
3. Enter a job name
4. Select 'Freestyle project'
5. Click OK
6. In 'Source Code Management', select Git and provide repository URL
7. In 'Build Environment', select required options if needed
8. In 'Build Steps', click 'Add build step' → 'Execute shell'
9. Enter build commands like 'mvn clean install'
10. Click Save
11. Click 'Build Now' to run the job

5. What is a Pipeline Job?

A Pipeline job is an advanced Jenkins job type that defines the entire build and deployment process as code using a Jenkinsfile. It provides better automation, version control, and complex workflow management.

6. Steps to Create a Pipeline Job

12. Open Jenkins dashboard
13. Click on 'New Item'
14. Enter a job name
15. Select 'Pipeline'
16. Click OK

17. Scroll to Pipeline section
18. Choose 'Pipeline script from SCM'
19. Select SCM as Git
20. Enter the GitHub repository URL
21. Specify the branch (e.g., main or master)
22. Set Script Path as 'Jenkinsfile'
23. Click Save
24. Click 'Build Now' to run the pipeline

7. Difference Between Freestyle and Pipeline Jobs

Freestyle Job	Pipeline Job
Configured via Jenkins UI	Defined as code in Jenkinsfile
Suitable for simple builds	Suitable for complex CI/CD workflows
Hard to version control	Easily version controlled with Git
Limited flexibility	Highly flexible and scalable

8. Conclusion

Freestyle jobs are useful for simple automation tasks, while Pipeline jobs are recommended for modern CI/CD practices as they allow full automation using code.

Implementation of CI/CD Pipeline for Java Web Application using Jenkins and Tomcat

1.Aim:

To automate the build and deployment process of a Java Maven web application using Jenkins and deploy it to Apache Tomcat server on AWS Ubuntu.

2.Tools & Technologies Required:

- Java 17 – Programming language runtime
- Apache Maven – Build and dependency management tool
- Jenkins – Continuous Integration/Deployment automation server
- Apache Tomcat – Web server to run WAR file
- GitHub – Source code repository
- AWS EC2 Ubuntu – Cloud server environment

3.Theory:

Continuous Integration (CI) is the process of automatically building and testing code whenever changes are made. Continuous Deployment (CD) is the process of automatically deploying the built application to a server. In this experiment, Jenkins is used to fetch source code from GitHub, Maven builds the application into a WAR file, and Tomcat deploys the application to be accessed through a browser.

4.Architecture Flow:

Developer → GitHub → Jenkins → Maven Build → WAR → Tomcat → Browser

5.Procedure:

Step 1: Install Java 17

```
sudo apt update
```

```
sudo apt install openjdk-17-jdk -y
```

```
java -version
```

Step 2: Install Maven

```
sudo apt install maven
```

To check version: `mvn -v`

Step 3: Install Jenkins

(for reference use Jenkins documentation)

```
sudo apt update
sudo apt install jenkins
sudo systemctl enable Jenkins
sudo systemctl start Jenkins
sudo systemctl status jenkins
```

To know running port number use: `sudo netstat -ltnp`

Access Jenkins at `http://<EC2-IP>:8080`

Step 4: Install and Configure Tomcat

```
sudo apt install tomcat10 tomcat10-admin -y
```

Here Jenkins and Tomcat both try to use port 8080 by default. They cannot run on the same port on the same server. So, we must separate the port. We will change Tomcat from 8080 → 8090. For this we edit the tomcat configuration below

->Change port from 8080 to 8090:

```
sudo nano /etc/tomcat10/server.xml
```

finds this line: `<Connector port="8080" protocol="HTTP/1.1"`

now, change to `<Connector port="8090" protocol="HTTP/1.1"`

For save the file use “ctrl+o ,next click “enter” button”

Quit from file use “ctrl+x”

```
sudo systemctl restart tomcat10
```

Step 5: Provide Deployment Permission

Tomcat owns the webapps folder, so Jenkins cannot copy files there.

We need to give Jenkins permission. means, Allow Jenkins to write into:
`/var/lib/tomcat10/webapps`.

Check folder owner by using this command: `ls -ld /var/lib/tomcat10/webapps`.

Now u will see something like this : `drwxr-xr-x tomcat tomcat webapps`

Add Jenkins User to Tomcat Group by follow below commands:

```
sudo usermod -aG tomcat jenkins
sudo chmod -R 775 /var/lib/tomcat10/webapps
```

Now, restart the Jenkins:

```
sudo systemctl restart jenkins
```

Step 6: Configure Jenkins Tools and add some plugins:

Manage Jenkins → Global Tool Configuration

To know java home path: `readlink -f $(which java)`

To know maven home path: `readlink -f $(which mvn)`

Add JDK path: `/usr/lib/jvm/java-17-openjdk-amd64`

Add Maven path: `/opt/maven`

And install below plugins:

- Git Plugin

- GitHub Plugin

- Pipeline Plugin, Pipeline Stage View

- Maven integration, git integration

Step 7: Create Jenkinsfile in GitHub

(Include pipeline script for checkout, build, archive, and deploy stages)

Fork my git repo as ur repo

My Git: <https://github.com/sampathiraoyoganandh/spring-boot-maven-example-helloworld.git>

Step 8: Create Jenkins Pipeline Job

New Item → Pipeline → Pipeline script from SCM → Provide GitHub repository URL
→ Save and Build

Step 9: Access Application

<http://<EC2-IP>:8090/SpringBootMavenExample/>

6.Result:

The Java web application was successfully built using Maven, automated through Jenkins pipeline, and deployed on Apache Tomcat server. The application was accessed successfully through a web browser.

7.Conclusion:

The CI/CD pipeline was successfully implemented. This experiment demonstrated how automation tools can reduce manual work and improve software delivery speed.